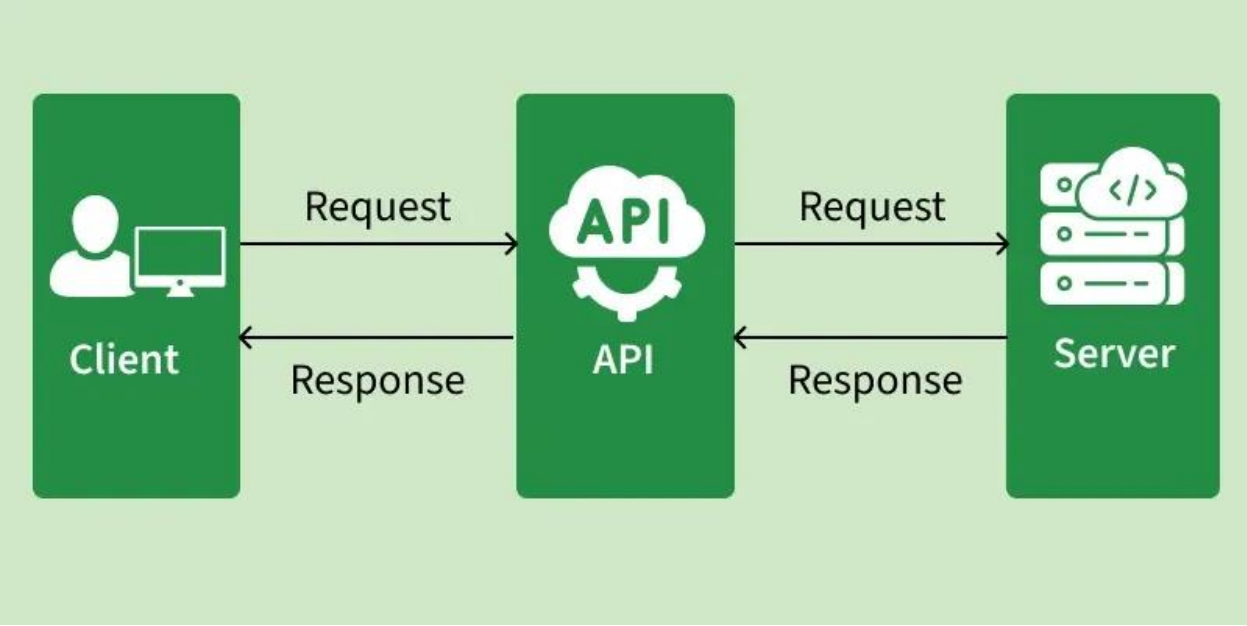
API (Application Programming Interface)

APIs, or Application Programming Interfaces, are the invisible backbone of modern software development. They enable applications and systems to communicate and share data efficiently.

- Enable seamless interaction between applications, platforms, and services.
- Power everyday features such as weather updates, payments, and e-commerce checkouts.

This is exactly how APIs work in software the API takes a request, sends it to the server, retrieves the data, and returns the response.



What is a Frontend API?

A frontend API, also known as a client-side API, connects an application frontend to external services or the backend. Its main goal is data interchange and interactive elements that improve UX. Frontend APIs focus on data presentation and real-time interaction, whereas backend APIs focus on server-side functionality.

Frontend APIs are essential for modern online and mobile apps. They let developers design dynamically updating responsive interfaces without page reloads. Single-page apps (SPAs) are popular owing to their fluid user experiences and ability to acquire and display data on demand. Frontend APIs allow developers to create native-like apps that offer quicker and more engaging experiences.

How Frontend APIs Interact with Backend APIs

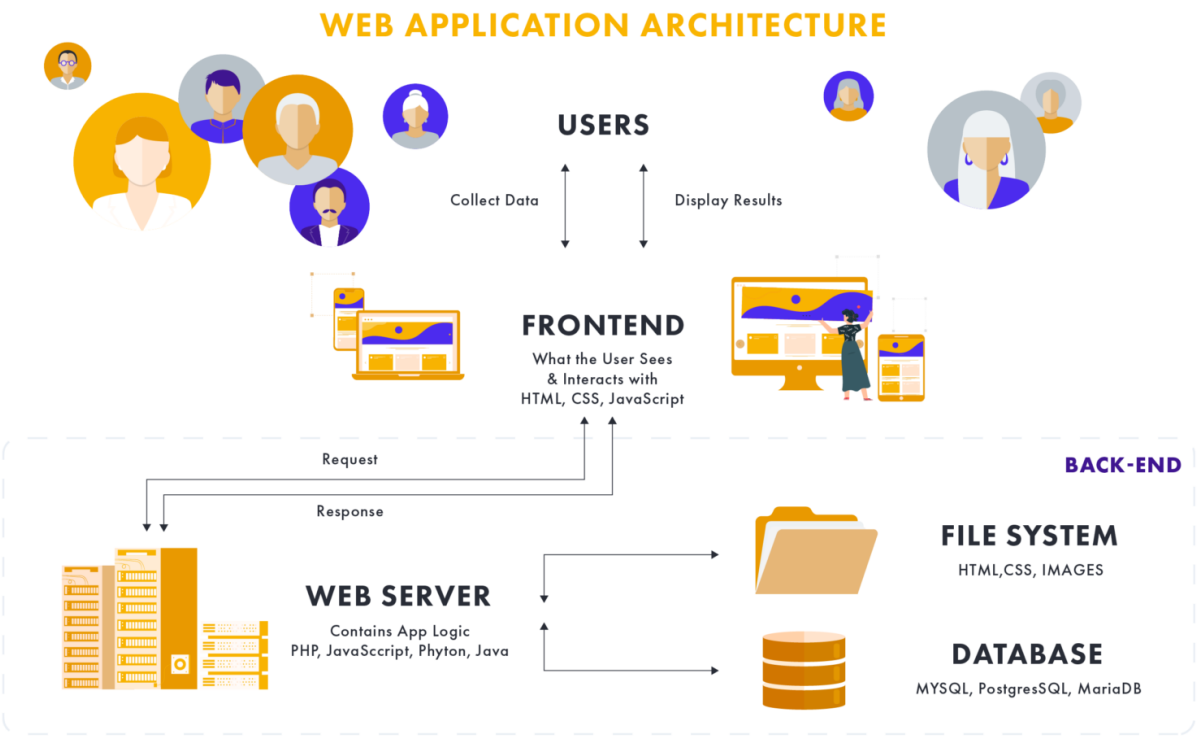
Frontend-backend API contact is essential to modern web development. User activities and server-side logic are connected using frontend APIs. These manage UI queries and backend API calls to get or alter data. Description of this relationship:

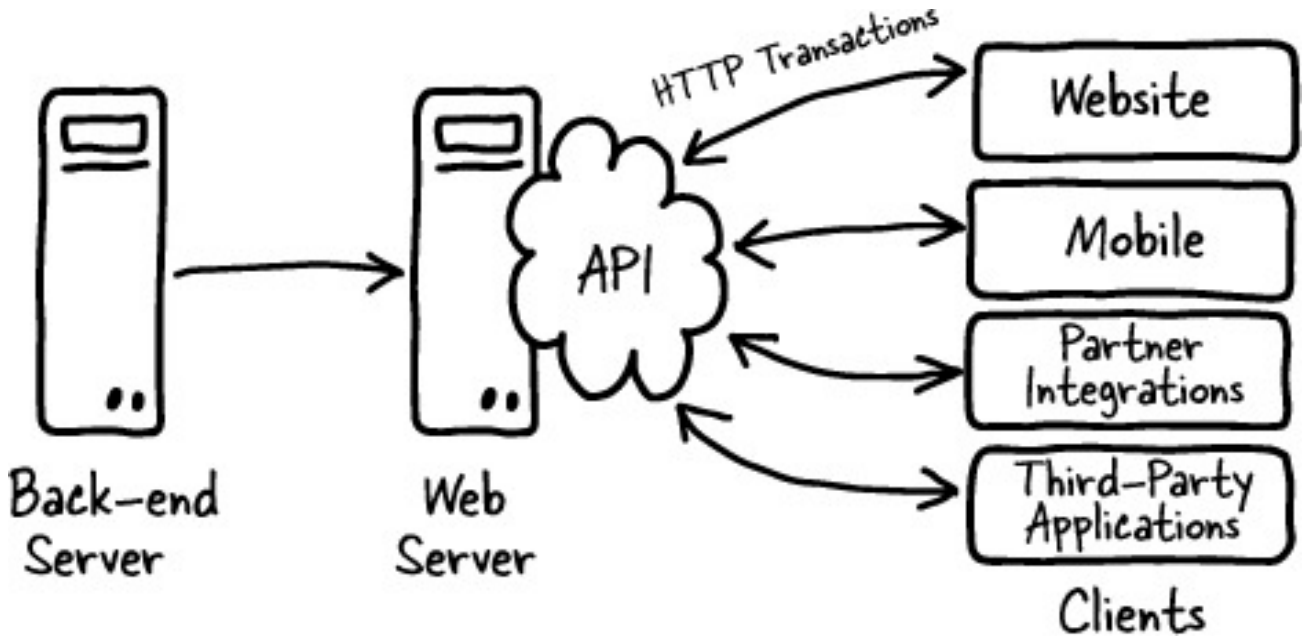
1. **Request Handling:** Frontend APIs send HTTP requests (GET, POST, PUT, DELETE) to backend APIs to get user data or submit forms.
2. **Data Manipulation:** The frontend API handles backend data and typically formats it for UI presentation.

3. **Dynamic Refreshes:** The frontend API refreshes the UI based on user interactions and backend answers for a seamless and responsive experience.

Key Differences Between Backend and Frontend APIs

Feature	Frontend APIs	Backend APIs
Location	Runs in the User's Browser/Device.	Runs on a Remote Server.
Access to Data	Can only access what the server "sends" it.	Has direct access to the Database.
Main Language	Primarily JavaScript/TypeScript.	Node.js, Python, Java, Go, Ruby, etc.
Key Concern	User Experience (UX) and performance.	Data Integrity, Security, and Scaling.
Visibility	Exposed to the end-user.	Encapsulated and private.


WEB APPLICATION ARCHITECTURE
USERS
Collect Data
Display Results
FRONTEND
What the User Sees
& Interacts with
HTML, CSS, JavaScript
Request
Response
BACK-END
WEB SERVER
Contains App Logic
PHP, JavaSccript, Phyton, Java
FILE SYSTEM
HTML,CSS, IMAGES
DATABASE
MYSQL, PostgresSQL, MariaDB


HTTP Transactions
Website
Mobile
Partner Integrations
Third-Party Applications
Clients
API
Back-end Server
Web Server

API Methods:

What are the REST APIs?

REST stands for Representational State Transfer, and follows the constraints of REST architecture allowing interaction with RESTful web services. It defines a set of functions (GET, PUT, POST, DELETE) that clients use to access server data. The functions used are:

- GET (retrieve a record)
- PUT (update a record)
- POST (create a record)
- DELETE (delete the record)

The 4 Primary API Methods

Method	Action	Simple Analogy
GET	**Retrieve** data	Like opening a book to read a specific page.
POST	**Create** data	Like adding a new entry to a diary.
PUT	**Update** data	Like replacing an old photo in a frame with a new one.
DELETE	**Remove** data	Like tearing a page out of a notebook.

1. GET: Retrieve Data

Purpose: Retrieve data from the server without modifying resources.

Best Practices:

- Use query parameters for filtering, sorting, and pagination.
- Avoid including sensitive data in URLs (e.g., passwords or API keys).
- Ensure GET requests are idempotent-repeating the same request should yield the same result.

Example:

```
GET /users/123 HTTP/1.1
Host: api.example.com
```

This request retrieves details for the user with ID 123.

2.POST: Create or Submit Data

Purpose: Create a new resource or submit data to the server.

Best Practices:

- Use POST for non-idempotent operations (e.g., creating a new user).
- Include proper error handling (e.g., return `400 Bad Request` for invalid input).
- Use the Location header to return the URL of the newly created resource.

Example:

```
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json

{
  "name": "John Doe",
  "email": "john.doe@example.com"
```

```
}
```

This request creates a new user with the provided details.

3.PUT: Update or Replace Resources

Purpose: Update or replace an existing resource.

Best Practices:

- Ensure PUT requests are idempotent—repeating the same request should have the same effect.
- Use PUT for full resource updates (e.g., replacing all fields of a user).
- Return `200 OK` for successful updates or `204 No Content` if no response body is needed.

Example:

```
PUT /users/123 HTTP/1.1
Host: api.example.com
Content-Type: application/json

{
  "name": "Jane Doe",
  "email": "jane.doe@example.com"
}
```

This request updates the user with ID 123, replacing their name and email.

4.DELETE: Remove Resources

Purpose: Remove a resource from the server.

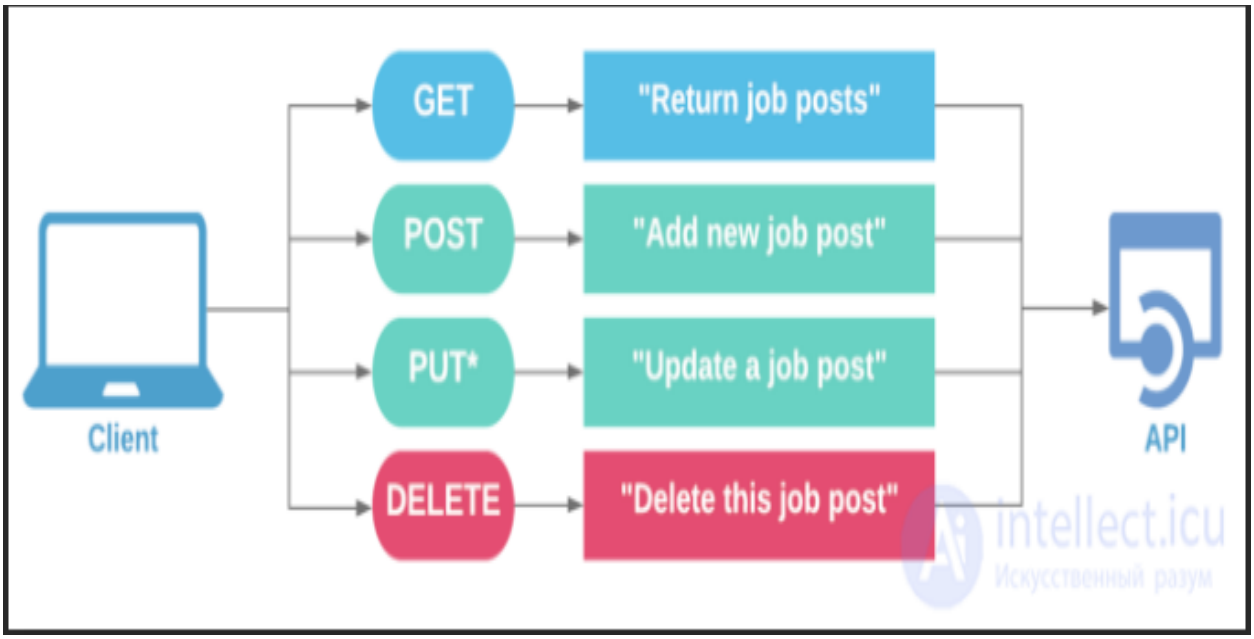
Best Practices:

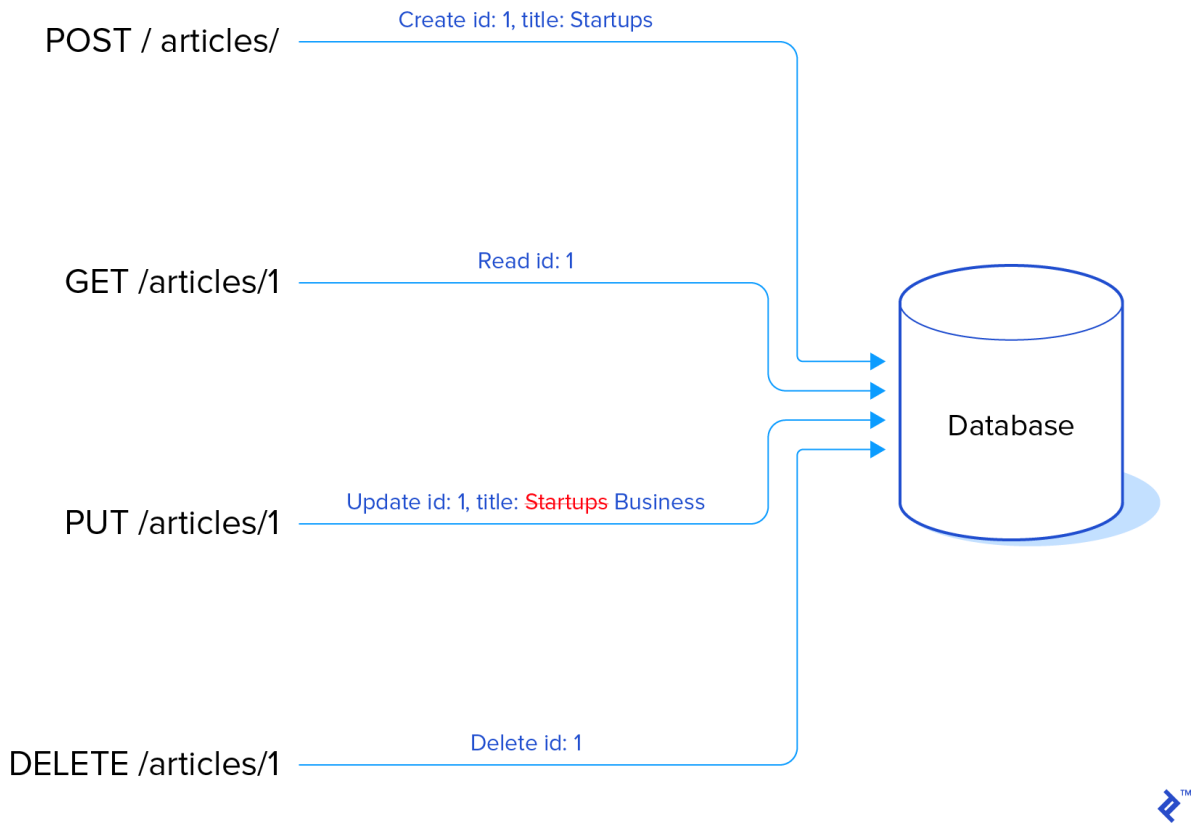
- Handle resource deletion gracefully (e.g., return `404 Not Found` if the resource doesn't exist).
- Ensure DELETE requests are idempotent—deleting a resource multiple times should have the same effect.
- Use `204 No Content` to indicate successful deletion.

Example:

```
DELETE /users/123 HTTP/1.1
Host: api.example.com
```

This request deletes the user with ID 123.





NAME: WIDAAD ALHANDASI.